UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA INFORMÁTICA



PROYECTO PARCIAL

Programación Orientada a Objetos

Fuga de la Prisión

Diseño e implementación de un videojuego 2D

Curso:	Programación Orientada a Objetos
Docentes:	MSc. Wilder Nina Choquehuayta (principal) MSc. Percy Lovón Ramos (auxiliar) Dr. Ortiz Ortiz Emanuel (auxiliar)
Tema asignado:	Fuga de la Prisión
Ciclo:	IV
Integrantes:	JOSE FERNANDO CHAVEZ LOZANO MICKELE CARLOS TORRES VELAZCO PIERO CEMEI ROMERO MEZA RICHARD AARON SORIANO CORDOVA
Período académico:	2026-II

Lima — Perú

2026

Informe parcial: Fuga de la Prisión

I. Resumen Ejecutivo

Fuga de la Prisión es un videojuego 2D en el que un prisionero busca una llave y escapa de la vigilancia. El avance actual incluye cuatro niveles, un jefe y 22 clases representadas en UML. La arquitectura separa personajes, reglas, estados, puntaje, combate y dibujo. El código demuestra asociación, agregación, composición, encapsulamiento, herencia y polimorfismo, junto con los patrones State y Observer. Las pruebas y las capturas permiten mostrar sus funciones actuales.

II. Introducción

a. Contexto y objetivo

El proyecto evalúa la aplicación de la Programación Orientada a Objetos mediante un videojuego. La temática asignada es Fuga de la Prisión: escapar evitando guardias, cámaras y sistemas de seguridad [1]. El objetivo de esta entrega parcial es demostrar un diseño de clases coherente, sus fundamentos POO y los patrones implementados, acompañados de una interfaz y funciones que se puedan ejecutar [2].

La versión actual tiene tres zonas de sigilo y una arena contra el Alcaide. Cada zona exige recoger su llave y llegar a la salida. En la arena se debe vencer al jefe antes de recoger la última llave. Esta descripción corresponde al código que acompaña la entrega.

b. Tecnologías utilizadas

Tecnología	Uso en el proyecto
Python y Pygame	Reglas del juego, ventana, teclado y dibujo de los personajes stickman. Se verificó con Python 3.13.7 y Pygame 2.6.1.
Mermaid.js	Fuente editable del UML y generación de la imagen del mismo modelo.
unittest	Comprobación de reglas, campaña, combate y coincidencia entre clases, UML y guía.
Git y GitHub	Registro y publicación de versiones. Los aportes se revisan mediante el historial real del repositorio.

III. Diseño de Arquitectura Conceptual y de Clases

a. Diagrama de clases UML

El diagrama incluye las 22 clases del juego definidas en src, conforme a la indicación del docente de representar todas. Cada caja tiene nombre, atributos propios y operaciones. Los tipos indican los objetos que guarda la clase: por ejemplo, guardias: list<Guardia>, carrera: HabilidadCarrera y jefe: Alcaide. Los miembros heredados se consultan en la clase base y se omiten los constructores para mantener legible el modelo.

El UML completo se incluye como última página ampliada del informe. Su fuente es modelo_parcial.mmd y la imagen modelo_parcial.png se genera desde ese archivo con Mermaid.js. Las clases de pruebas no son componentes del videojuego.

b. Estructura de componentes

Archivo y clases	Responsabilidad
juego.py: Juego	Coordina la campaña, carga niveles, conserva puntos y delega en el estado actual.
estados.py: EstadoJuego, Jugando, Pausa, Derrota, Victoria, NivelCompletado	Definen cómo responde la partida a las entradas y cuándo se actualiza el nivel.
nivel.py: Nivel, Camara, Llave, PuertaSalida	Reúnen mapa, vigilancia fija, objetivo y salida; resuelven detección, premios y fin de nivel.
personajes.py: Personaje, Prisionero, Guardia, HabilidadCarrera	Comparten movimiento y colisiones; separan el control del jugador, la patrulla y la carrera.
puntuacion.py: Puntaje, ObservadorPuntaje, PanelPuntaje	Validan los puntos y comunican sus cambios al panel mediante Observer.
combate.py: Proyectil, Alcaide, Combate	Gestionan movimiento de disparos, ataques del jefe, impactos, vida e inmunidad.
vista.py: Vista	Dibuja la interfaz, el mapa, los personajes y los estados sin resolver las reglas del juego.

Lectura de las relaciones

El rombo negro representa composición y queda junto al objeto que crea sus partes; el blanco representa agregación. El triángulo vacío apunta a la clase base de una herencia. La flecha continua representa una referencia guardada y la discontinua un uso o dependencia. Las cantidades 1, 0..1 y 0..* significan uno, ninguno o uno, y ninguno o varios. El signo + indica acceso público; # señala uso interno por convención de Python.

IV. Aplicación de Fundamentos de POO

a. Asociación, agregación y composición

Asociación. Personaje recibe su Nivel y conserva la referencia para consultar muros, tiempo y entradas. No crea otro nivel:

```
self.nivel = nivel
```

Agregación. Juego crea Puntaje y lo entrega al nuevo Nivel. Al cambiar o repetir un nivel se conserva ese objeto; solo se ajusta su valor cuando corresponde:

```
self.puntaje = Puntaje()
self.nivel = Nivel(self.puntaje, self.indice_nivel)
```

Composición. Nivel crea sus propios personajes, cámaras, llave y salida. En el nivel final también crea Combate. Prisionero crea su habilidad y Combate crea al jefe y a los proyectiles. Los atributos del UML muestran las clases de esas partes:

```
self.carrera = HabilidadCarrera()
self.jefe = Alcaide()
```

La primera línea pertenece a Prisionero y la segunda a Combate. Combate.emitir crea cada Proyectil; su código completo aparece en la guía de clases.

b. Herencia y polimorfismo

Prisionero y Guardia heredan de Personaje. Comparten posición y movimiento, pero cada uno implementa update: el prisionero responde al teclado y el guardia sigue su ruta. El ciclo del nivel llama al mismo método sin distinguir el tipo de personaje:

```
for personaje in self.personajes:
    personaje.update()
```

También hay polimorfismo en los estados: Juego llama a actualizar y manejar_entrada del EstadoJuego activo. Los cinco estados concretos responden según la situación. PanelPuntaje hereda de ObservadorPuntaje e implementa actualizar_puntaje.

c. Encapsulamiento y abstracción

Puntaje guarda _valor, lo consulta con get_valor y lo modifica mediante set_valor, sumar y reiniciar. El setter rechaza datos que no sean enteros no negativos y notifica el cambio. Personaje consulta su posición mediante get_posicion y la cambia con mover, que comprueba los muros. El guion bajo indica uso interno; en Python no impide por sí solo el acceso directo.

Personaje, EstadoJuego y ObservadorPuntaje son abstractas: definen operaciones que las clases hijas deben implementar. Esto permite trabajar con una base común y conservar el comportamiento propio de cada objeto.

Patrones de diseño implementados

State. EstadoJuego es la base de Jugando, Pausa, Derrota, Victoria y NivelCompletado. Juego delega la entrada y actualización en el estado activo. La pausa detiene personajes, vigilancia, temporizadores y combate. NivelCompletado espera Enter antes de cargar el siguiente mapa.

Observer. Juego suscribe PanelPuntaje a Puntaje. Cada cambio válido notifica a los observadores mediante actualizar_puntaje. Vista lee el valor recibido por el panel para dibujarlo; Puntaje no depende de la ventana gráfica.

VI. Implementación de Resultados Parciales

La campaña incluye Pabellón de celdas, Lavandería, Patio de vigilancia y El Alcaide. Se puede mover con WASD o flechas, correr con Shift, recoger llaves, escapar y cambiar de nivel. En la arena, Espacio apunta al jefe e IJKL fija la dirección del disparo. El jefe avisa antes de atacar y tiene dos fases.

La vigilancia combina conos frontales con una alerta de 32 píxeles alrededor de cada guardia. Acercarse por el costado o por detrás también puede provocar la captura. La detección considera el cuerpo del jugador y los muros bloquean ambos campos. El círculo dibujado y la orientación de las cámaras permiten reconocer las zonas peligrosas.



Figura 1. Interfaz actual: objetivo, puntos, tiempo, carrera, vigilancia, llave y salida. Captura generada por el renderizador del juego.

Resultados comprobados y uso de Git

El 15 de octubre de 2026 se ejecutaron 40 pruebas automáticas sin fallos. Cubren colisiones, movimiento, vigilancia cercana y frontal, cobertura de muros, estados, puntos, transiciones, combate y correspondencia del UML y la guía con el código. La verificación completó los cuatro niveles mediante movimiento y disparos, sin desactivar enemigos, y terminó con 3400 puntos. Se generaron diez capturas.

La llave concede 100 puntos, cada salida 500 y el jefe 1000. Al perder, R repite el nivel y restaura el puntaje de entrada; N comienza desde cero. La rúbrica pide demostrar un avance del 25 % con interfaces y funciones [2], pero no establece una fórmula para medirlo. Aquí se presentan resultados que pueden ejecutarse y comprobarse.

Git. Hasta fc416a6, el historial revisado contiene once commits: nueve con el autor configurado JhosFcL, uno con Simei y uno con shzjeud. Las versiones registran planificación, implementación, documentación y aportes posteriores. Estos nombres no identifican por sí solos el trabajo de los cuatro integrantes; cada miembro debe explicar sus aportes reales. El detalle se conserva en docs/evidencias/git_estado.md.

VII. Conclusiones

Se dispone de cuatro niveles con movimiento, colisiones, vigilancia, llaves, salidas, puntaje y combate final. Las 22 clases están representadas en el UML y explicadas con su código actual. Se pueden demostrar los seis fundamentos POO solicitados, State en los cambios de situación y Observer al actualizar los puntos.

Una dificultad fue hacer coherentes la vigilancia visible y la detección, sin que los muros dejasen pasar al jugador o la visión. Las pruebas comprobaron la alerta lateral, la cobertura y una ruta completa. Separar reglas y dibujo facilitó estas correcciones. El equipo debe ensayar con teclado y explicar su trabajo en la exposición.

VIII. Anexos

Fragmento 1. Delegación de Juego al estado activo:

```
def manejar_entrada(self, tecla):
    self.estado.manejar_entrada(tecla)
```

Fragmento 2. Notificación de Puntaje a sus observadores:

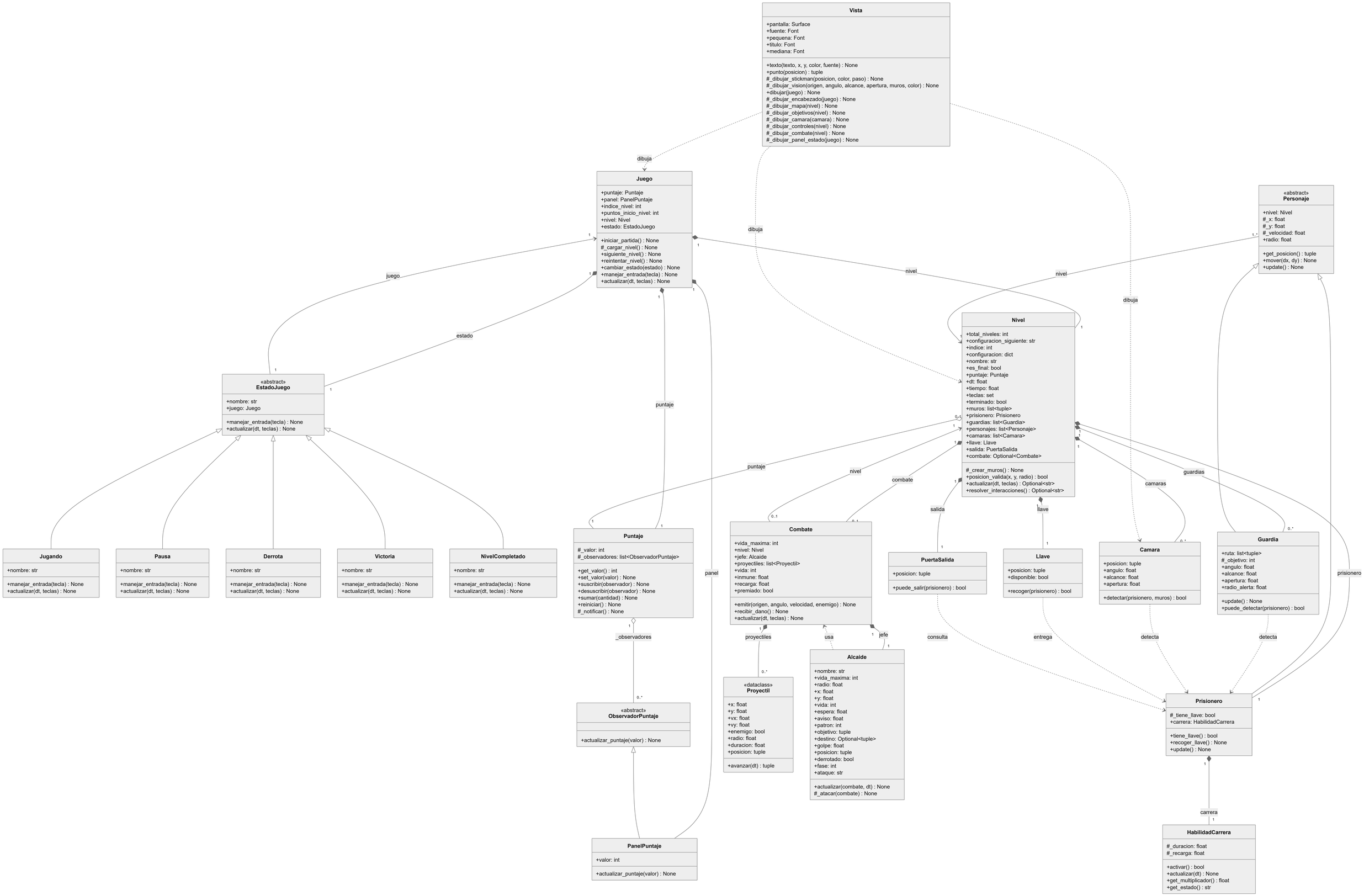
```
def _notificar(self):
    for observador in tuple(self._observadores):
        observador.actualizar_puntaje(self._valor)
```

El diagrama completo está en la página ampliada siguiente. La guía Explicacion_de_clases.md reúne las 22 clases y sus bloques completos. Las capturas y la salida reproducible de las pruebas se encuentran en docs/evidencias.

Material del curso

[1] Proyecto Final POO, ciclo 2026-2, temática 13.
[2] Rúbrica y Estructura - Proyecto Parcial, ciclo 2026-2.

VIII. Anexos - Diagrama UML completo: 22 clases del juego



Tres partes por clase: nombre, atributos propios y operaciones. Las propiedades de consulta aparecen como atributos. Los miembros heredados se leen en la clase base.

Rombo negro: composición · Rombo blanco: agregación · Triángulo: herencia · Flecha continua: asociación · Flecha discontinua: dependencia

Fuente editable: modelo_parcial.mmd · Imagen generada con Mermaid.js · Página 7